Emberbound — Milestone 2 Independent Audit

Tap Upgrades, Economy & BigNum • PR #1 • 31 October 2026

Executive verdict

FAIL — PR #1 should remain unmerged until the BigNum correctness issues below are corrected. The existing tests pass, but additional checks reproduce violations of the exact-small-result and non-throwing-parser contracts. I found no demonstrated client-controlled pricing, negative-Gold, replay double-charge, or cross-player exploit.

Reviewed actual GitHub PR head `eb5a3511d932de5c1860852c0f47dfa8911912a5` against `main` at `730218a`. Its source matches implementation commit `024cd68838d6c34633a00e7eadcb320c251186f0`.

All findings below were confirmed from source and reproduced with Luau 0.745. **No Roblox Studio tests were executed.**

Findings table

F1 | Medium — required correctness fix

Location: src/ReplicatedStorage/Shared/BigNum.luau:263–278, `sub()`

Defect, reproduction, and impact: Converting operands to scientific notation loses precision during cancellation, including when the result returns below `2^53`. `sub(fromNumber(9007199254740992), fromNumber(9007199254740991))` returns `1.7763568394002502`, rather than `1`. `Wallet.trySpend()` propagates this into balances. With Gold `2^53`, spending the actual level-510 cost produces `4485383672432620.5`, rather than the exact integer `4485383672432620`.

Minimal recommended correction: Add a checked native-number subtraction path where operands fit and the result returns to the exact range. Add cross-regime subtraction and wallet-conservation tests.

F2 | Medium — required correctness fix

Location: src/ReplicatedStorage/Shared/BigNum.luau:401–435, `parse()`

Defect, reproduction, and impact: The parser checks the supplied exponent before normalization, then calls a normalizer that can throw. `parse("10e9007199254740991")` raises an overflow error instead of returning `nil`. This breaks its documented validation contract and can interrupt callers processing malformed serialized values.

Minimal recommended correction: Validate the normalized exponent or catch normalization overflow and return `nil`. Add mantissa-plus-exponent overflow tests.

F3 | Low

Location: src/ServerScriptService/Tests/TestRunner.luau:29–39, suite loading

Defect, reproduction, and impact: Failed suite loads produce a warning and are omitted from the result. Injecting a load failure for `BigNumTests` still produces **“ALL PASSED (27 tests)”**. The offline wrapper accepts this success message. Broken or missing suites can therefore yield a false green result.

Minimal recommended correction: Treat suite-load failures as test failures; reject an empty run and propagate failure through the runner.

F4 | Low

Location: src/ReplicatedStorage/Shared/BigNum.luau:80–92, `toScientific()`

Defect, reproduction, and impact: Accepted subnormal values can break arithmetic. `mul(fromNumber(5e-324), parse("1e400"))` throws an overflow error despite a representable result. The conversion divides by `10^-324`, which underflows to zero. Current integer-based M2 formulas do not generate this input.

Minimal recommended correction: Normalize subnormals using safe intermediate scaling; test arithmetic on the smallest accepted positive values.

F5 | Low

Location: src/ReplicatedStorage/Shared/BigNum.luau:22–37, exported type and `make()`

Defect, reproduction, and impact: Independently reproduced the known strict-type diagnostic: `table.freeze()` returns read-only fields, while exported `BigNum` requires writable fields. Runtime construction succeeds, but strict analysis remains unclean.

Minimal recommended correction: Make the exported type compatible with the intended immutable representation.

F6 | Low

Location: src/ServerScriptService/Tests/TestKit.luau:47–50, `expectNear()`

Defect, reproduction, and impact: `expectNear(0/0, 123, 0)` **passes.** Comparisons against NaN are false, so numerical failures can escape approximate assertions.

Minimal recommended correction: Explicitly reject NaN inputs and invalid tolerances before comparing the difference.

No Critical-severity defect was identified. **F1 and F2 are the reasons I recommend withholding approval:** they contradict explicit requirements of the numerical foundation.

Economy and security assessment

The current purchase path has several sound properties:

- Prices, levels, and damage come from server-owned configuration.
- Spending, level mutation, damage application, and reply construction contain no yield.
- Request IDs increase per player; replayed and older IDs cannot charge again.
- Rate limiting occurs before purchase calculation.

- In the supplied configuration, the combat-state check and non-yielding execution prevent ordinary player removal from interrupting the purchase between spending and applying its effect.
- Wallet spending checks affordability first and never produces a negative balance.

Although `applyUpgradeEffect()`'s return value is ignored, I found no reachable failure-after-charge path under the current single-upgrade configuration. That assumption needs renewed testing when M2.1 changes the effect destination.

Precision absorption is economically negligible under the current curve. Starting with `1e400` Gold, I reproduced 13,036 purchases before the first visible deduction. Their combined ignored costs were approximately `7.96e384`, or `7.96×10^-16` **of the balance**. This supports accepting the documented approximation for M2. F1 is a separate cancellation problem that exposes inaccurate small residual balances.

Automated test assessment

Independently executed check	Result
Source checksums	**23/23 match the manifest**
Luau compilation	**23/23 source files compile**
Shipped unit suites	**48 passed, 0 failed**
Existing review probes	**54 passed, 0 failed**
Focused boundary and fault-injection checks	Reproduced F1–F4 and F6
Strict analysis of BigNum	Reproduced F5
Selene lint	Not independently rerun
Roblox Studio/client execution	Not performed

The existing tests provide useful coverage of formulas, normal arithmetic, spending, replay protection, rate limits, and two-player server isolation. They miss cross-regime cancellation, normalization overflow during parsing, subnormal arithmetic, and test-loader failure reporting.

The fake engine also executes handlers synchronously, never waits for replication, and does not run the client. Its passing results do not establish GUI correctness, deferred-event behavior, device performance, or Studio source parity.

Synchronization and Milestone 1 regression assessment

Source review preserves the important M1 behavior:

- `ArenaView` is unchanged, including `Camera.Focus` initialization and rebinding.
- The final three-second synchronization response window remains present.
- Server tap cooldown, per-player combat state, stage progression, and the central damage path remain intact.
- Defeat handling replaces the enemy without yielding, preventing duplicate rewards for that enemy.
- The revised client consistently decodes BigNum strings and obtains Gold through `CurrencyUpdate`.

- Service startup has no require cycle; per-player state and limiter entries have removal paths.
- I found no unbounded connection or transient-object retention path in the intended once-mounted client lifecycle.

The upgrade panel's rectangle exclusion supplements processed-input filtering. Its coordinate comparison is consistent with the input coordinate approach already reviewed in M1. Actual mobile layout and touch behavior still require Studio testing.

Suggested Roblox Studio regression tests

After the accepted corrections:

1. **Source parity and startup:** compare all 23 installed scripts with the reviewed revision, create the six documented remotes, and require all expected suites to execute successfully.
2. **M1 regression:** verify ten accepted taps yield one stage-1 defeat, 5 Gold, and the stage-2 enemy at 11 HP. Repeat fixed-camera and delayed-sync checks.
3. **Purchases:** verify Gold, level, HUD damage, panel damage, and actual hit damage agree after each purchase, including level 24→25.
4. **Request abuse:** exercise rapid, replayed, malformed, and out-of-order requests while confirming affordability and player isolation.
5. **Numerical boundaries:** exercise wallets around `2^53`, serialized normalization overflow, and values beyond `1e308`.
6. **Latency:** make several purchases before replies arrive; verify final balances, upgrade views, and combat stats converge correctly.
7. **Mobile UI:** test portrait/landscape phones and tablets, safe-area insets, long scientific-number labels, and touch followed by mouse input. Panel presses must never attack.
8. **Cleanup:** repeat joins/leaves and sustained combat while watching server state, retired enemies, transient GUI objects, and memory.

Milestone 2.1 / 3 readiness

The current interfaces can support the approved StatService and later persistence without an immediate architecture rewrite. Direct damage replacement is the explicitly approved M2.1 follow-up, not an additional M2 defect.

Recommended sequence: correct F1/F2, repair the test-reporting weaknesses, rerun the numerical and service checks, then complete Studio regression testing. Resolve F4/F5 before treating BigNum as a generally validated shared numerical API.

For M3, add the already planned profile validation, restore ordering, dirty tracking, and cancellation safeguards around asynchronous loading. Request IDs currently provide session-local replay protection.

Do not merge PR #1 as-is, even if basic Studio gameplay passes. After the focused corrections and successful Studio regression results, I recommend merging and proceeding to M2.1.

The checkout's source files remain unchanged. No commits, pushes, PR merge, or environment publication were performed.